



Treehouse tAVAX BenqiStrategy Audit Report

Oct 22, 2025





Table of Contents

Summary	2
Overview	3
Issues	4
[WP-C1] <code>NavBenqi</code> and <code>NavErc20</code> Double-Count Native Token (AVAX) Balance in Total NAV, Leading to Inflated NAV	4
[WP-H2] Rewards claimed via <code>claimReward()</code> by external parties are not properly tracked.	8
[WP-M3] <code>NavBenqi</code> fails to handle interest accrued on BENQI between last settlement and current moment	12
[WP-L4] Replace imprecise <code>exchangeRateStored()</code> redemption calculation with direct <code>redeemUnderlying()</code> call	17
[WP-L5] <code>NavBenqi</code> does not handle error codes returned by <code>IBENQI.getAccountSnapshot()</code> , potentially misusing invalid values	19
Appendix	22
Disclaimer	23



Summary

This report has been prepared for Treehouse smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.



Overview

Project Summary

Project Name	Treehouse
Codebase	https://github.com/treehouse-gaia/tAVAX-protocol
Commit	9ce06b363f33c0b86535cd625a9dfb5d21d90e2d
Language	Solidity

Audit Summary

Delivery Date	Oct 22, 2025
Audit Methodology	Static Analysis, Manual Review
Total Issues	5

[WP-C1] NavBenqi and NavErc20 Double-Count Native Token (AVAX) Balance in Total NAV, Leading to Inflated NAV

Critical

Issue Description

In `navRegistry`, `newStrat` has both `NavErc20` and `NavBenqi` modules.

Each module includes `newStrat.balance` (AVAX balance) as part of their returned `_nav`. Since `NavRegistry.getStrategyNav()` at L240 and L252 accumulates `_nav` values from all modules, `newStrat.balance` is counted twice.

Attackers can inflate the share price by transferring native tokens to the contract and redeem at a inflated rate. By repeating this process, attackers can drain the protocol.

https://github.com/treehouse-gaia/tAVAX-protocol/blob/181a25d9ad126b13eabb9957ba3e7f828b5a6e23/scripts/deployment_3/1_deploy-benqi-action.ts#L189-L200

```

189     const benqi_cd = await navBenqi.nav.populateTransaction(newStrat,
[CONSTANTS.QISAVAX],[CONSTANTS.QIAVAX])
190     const erc20_cd = await navErc20.nav.populateTransaction(newStrat, [
191         CONSTANTS.SAVAX,
192         CONSTANTS.WAVAX]
193     )
194     const unsavax_cd = await navUnSavax.nav.populateTransaction(newStrat)
195     // attach nav modules and calldata to strat
196     console.log(`=== Attaching nav modules ===`)
197     // await navRegistry.attachTo(newStrat, { moduleId:
getActionIdFromName(`NavErc20WithDebt`), cd: erc20withdebt_cd.data }) //
NavErc20WithDebt register
198     txs.push(navRegistry.attachTo.populateTransaction(newStrat, { moduleId:
getActionIdFromName(`NavErc20`), cd: erc20_cd.data }))
199     txs.push(navRegistry.attachTo.populateTransaction(newStrat, { moduleId:
getActionIdFromName(`NavUnSavax`), cd: unsavax_cd.data }))
200     txs.push(navRegistry.attachTo.populateTransaction(newStrat, { moduleId:
getActionIdFromName(`NavBenqi`), cd: benqi_cd.data }))

```

<https://github.com/treehouse-gaia/tAVAX-protocol/blob/181a25d9ad126b13eabb9957ba3e7f828b5a6e23/contracts/NavRegistry.sol#L214-L263>

```

214  /**
    @@ 215,222 @@
223  * @return _navInUnderlying sum of all attached modules
224  */
225  function getStrategyNav(
226      address strategy,
227      ModuleParams[] calldata dynamicModuleParams
228  ) external view returns (uint _navInUnderlying) {
229      uint256 moduleIdsLength = _strategyModuleIds[strategy].moduleIds.length();
230      for (uint i; i < moduleIdsLength; ++i) {
231          bytes4 moduleId = bytes4(_strategyModuleIds[strategy].moduleIds.at(i));
232          if (bytes32(strategyModuleCd[strategy][moduleId]) != DYNAMIC) {
233              (bool success, bytes memory info) =
234              address(modules[moduleId].addr).staticcall(
235                  strategyModuleCd[strategy][moduleId]
236              );
237              if (!success) revert GetNavFailed(moduleId);
238              unchecked {
239                  _navInUnderlying += uint(bytes32(info));
240              }
241          } else {
242              // get first instance of dynamic cd
243              bool executed = false;
244              for (uint j; j < dynamicModuleParams.length; ++j) {
245                  if (dynamicModuleParams[j].moduleId == moduleId) {
246                      (bool success, bytes memory info) =
247                      address(modules[moduleId].addr).staticcall(dynamicModuleParams[j].cd);
248                      if (!success) revert GetNavFailed(moduleId);
249                      unchecked {
250                          _navInUnderlying += uint(bytes32(info));
251                      }
252                      executed = true;
253                      break;
254                  }
255              }
256          }
257      }

```

```

258     }
259
260     if (!executed) revert MissingDynamicModule(moduleId);
261 }
262 }
263 }

```

```

@@ 32,40 @@
41     function nav(
42         address _target,
43         address[] memory _assetTokens,
44         address[] memory _debtTokens
45     ) external view returns (uint _nav) {
46         _nav += _target.balance;
47
@@ 48,83 @@
84
85     if (_nav == 0) {
86         return 0;
87     } else {
88         // Handle potential revert from getSharesByPooledAvax when amount is too small
89         try sAVAX.getSharesByPooledAvax(_nav) returns (uint avaxShares) {
90             _nav = avaxShares;
91         } catch {
92             // If conversion fails (likely due to amount being too small), just return 0
93             _nav = 0;
94         }
95     }
96 }

```

```

@@ 25,32 @@
33     function nav(address _target, address[] memory _tokens) external view returns
34         (uint _nav) {
35         _nav += _target.balance;

```



```
@@ 36,52 @@  
53     if (_nav == 0) {  
54         _nav = sAVAXBalace;  
55     } else {  
56         // Handle potential revert from getSharesByPooledAvax when amount is too  
small  
57         try sAVAX.getSharesByPooledAvax(_nav) returns (uint avaxShares) {  
58             _nav = avaxShares + sAVAXBalace;  
59         } catch {  
60             // If conversion fails (likely due to amount being too small), just return  
sAVAX balance  
61             _nav = sAVAXBalace;  
62         }  
63     }  
64 }
```

Status

✓ Fixed

[WP-H2] Rewards claimed via `claimReward()` by external parties are not properly tracked.

High

Issue Description

The current implementation of `BenqiClaimReward.sol` only tracks reward amounts that are directly claimed by the contract.

Benqi's protocol allows any address to claim rewards on behalf of other users.

When rewards are claimed by external parties, they are not forwarded to the TREASURY or converted to wAVAX.

Additionally, the current contracts and actions cannot handle native AVAX tokens.

<https://github.com/treehouse-gaia/tAVAX-protocol/blob/181a25d9ad126b13eabb9957ba3e7f828b5a6e23/contracts/strategy/actions/benqi/BenqiClaimReward.sol#L62-L101>

```

62     function _claimReward(
63         uint8 _rewardType,
64         address payable[] memory _holders,
65         address[] memory _qiTokens,
66         bool _borrowers,
67         bool _suppliers
68     ) internal returns (uint, bytes memory) {
69         // Validate inputs
70         require(_rewardType <= 1, 'Invalid reward type');
71         require(_holders.length > 0, 'No holders specified');
72         require(_qiTokens.length > 0, 'No qiTokens specified');
73
74         // Get comptroller address dynamically from qiAVAX contract
75         address comptrollerAddr = IBENQI(qiAVAX).comptroller();
76
77         uint qiBalanceBefore = IERC20(qi).balanceOf(address(this));
78         uint avaxBalanceBefore = address(this).balance;
79
80         // Claim rewards from the comptroller

```

```

81     IComptroller(comptrollerAddr).claimReward(_rewardType, _holders, _qiTokens,
    _borrowers, _suppliers);
82
83     if (_rewardType == 0) {
84         // If QI rewards were claimed, transfer them to the treasury
85         uint qiBalanceAfter = IERC20(qi).balanceOf(address(this));
86         uint qiBalanceClaimed = qiBalanceAfter - qiBalanceBefore;
87         if (qiBalanceClaimed > 0) {
88             IERC20(qi).safeTransfer(TREASURY, qiBalanceClaimed);
89         }
90     } else if (_rewardType == 1) {
91         // If AVAX rewards were claimed, wrap them to WAVAX
92         uint avaxBalanceAfter = address(this).balance;
93         uint avaxBalanceClaimed = avaxBalanceAfter - avaxBalanceBefore;
94         if (avaxBalanceClaimed > 0) {
95             IWAVAX(wAVAX).deposit{ value: avaxBalanceClaimed }();
96         }
97     }
98
99     bytes memory logData = abi.encode(comptrollerAddr, _rewardType, _holders,
    _qiTokens);
100    return (0, logData);
101 }

```

<https://github.com/Benqi-fi/BENQI-Smart-Contracts/blob/e0cfd244726719dfe027c9740878d64d1cad98f2/lending/Comptroller.sol#L1230-L1278>

```

1230    /**
1231     * @notice Claim all the benqi accrued by holder in all markets
1232     * @param holder The address to claim BENQI for
1233     */
1234    function claimReward(uint8 rewardType, address payable holder) public {
1235        return claimReward(rewardType, holder, allMarkets);
1236    }
1237
1238    /**
1239     * @notice Claim all the benqi accrued by holder in the specified markets
1240     * @param holder The address to claim BENQI for
1241     * @param qiTokens The list of markets to claim BENQI in
1242     */
1243    function claimReward(uint8 rewardType, address payable holder, QiToken[]
    memory qiTokens) public {

```

```

1244     address payable [] memory holders = new address payable[](1);
1245     holders[0] = holder;
1246     claimReward(rewardType, holders, qiTokens, true, true);
1247 }
1248
1249 /**
1250  * @notice Claim all Benqi or Avax accrued by the holders
1251  * @param rewardType 0 means Qi 1 means Avax
1252  * @param holders The addresses to claim AVAX for
1253  * @param qiTokens The List of markets to claim AVAX in
1254  * @param borrowers Whether or not to claim AVAX earned by borrowing
1255  * @param suppliers Whether or not to claim AVAX earned by supplying
1256  */
1257 function claimReward(uint8 rewardType, address payable[] memory holders,
QiToken[] memory qiTokens, bool borrowers, bool suppliers) public payable {
1258     require(rewardType <= 1, "rewardType is invalid");
1259     for (uint i = 0; i < qiTokens.length; i++) {
1260         QiToken qiToken = qiTokens[i];
1261         require(markets[address(qiToken)].isListed, "market must be listed");
1262         if (borrowers == true) {
1263             Exp memory borrowIndex = Exp({mantissa: qiToken.borrowIndex()});
1264             updateRewardBorrowIndex(rewardType, address(qiToken), borrowIndex);
1265             for (uint j = 0; j < holders.length; j++) {
1266                 distributeBorrowerReward(rewardType, address(qiToken),
holders[j], borrowIndex);
1267                 grantRewardInternal(rewardType, holders[j],
rewardAccrued[rewardType][holders[j]]);
1268             }
1269         }
1270         if (suppliers == true) {
1271             updateRewardSupplyIndex(rewardType, address(qiToken));
1272             for (uint j = 0; j < holders.length; j++) {
1273                 distributeSupplierReward(rewardType, address(qiToken),
holders[j]);
1274                 grantRewardInternal(rewardType, holders[j],
rewardAccrued[rewardType][holders[j]]);
1275             }
1276         }
1277     }
1278 }

```



Status

✓ Fixed

[WP-M3] NavBenqi fails to handle interest accrued on BENQI between last settlement and current moment

Medium

Issue Description

Unlike AAVE's `aToken.balanceOf()` and `debtToken.balanceOf()`, `IBENQI.getAccountSnapshot()` does not calculate interest accrued between last settlement and current moment - it directly uses the storage value.

<https://github.com/treehouse-gaia/tAVAX-protocol/blob/181a25d9ad126b13eabb9957ba3e7f828b5a6e23/contracts/modules/nav/NavBenqi.sol#L32-L96>

```

@@ 32,40 @@
41     function nav(
42         address _target,
43         address[] memory _assetTokens,
44         address[] memory _debtTokens
45     ) external view returns (uint _nav) {
46         _nav += _target.balance;
47
48         uint wip;
49
50         // Process asset tokens
51         for (uint i; i < _assetTokens.length; ++i) {
52             (, uint qiTokenBalance, , uint exchangeRate) =
53             IBENQI(_assetTokens[i]).getAccountSnapshot(_target);
54             wip = (qiTokenBalance * exchangeRate) / 1e18;
55
56             if (wip > 0) {
57                 uint rate = RATE_PROVIDER_REGISTRY.getRateInAvax(_assetTokens[i]);
58                 if (rate > 0) {
59                     _nav += Math.mulDiv(rate, wip, 1e18);
60                 }
61             }
62
63             // Process debt tokens - accumulate total debt first
64             uint totalDebt;

```

```

65     for (uint i; i < _debtTokens.length; ++i) {
66         (, , uint borrowBalance, ) =
IBENQI(_debtTokens[i]).getAccountSnapshot(_target);
67         wip = borrowBalance;
68
69         if (wip > 0) {
70             uint rate = RATE_PROVIDER_REGISTRY.getRateInAvax(_debtTokens[i]);
71             if (rate > 0) {
72                 totalDebt += Math.mulDiv(rate, wip, 1e18);
73             }
74         }
75     }
76
@@ 77,95 @@
96     }

```

```

197     /**
198     * @notice Get a snapshot of the account's balances, and the cached exchange
rate
199     * @dev This is used by comptroller to more efficiently perform liquidity
checks.
200     * @param account Address of the account to snapshot
201     * @return (possible error, token balance, borrow balance, exchange rate
mantissa)
202     */
203     function getAccountSnapshot(address account) external view returns (uint,
uint, uint, uint) {
204         uint qiTokenBalance = accountTokens[account];
205         uint borrowBalance;
206         uint exchangeRateMantissa;
207
208         MathError mErr;
209
210         (mErr, borrowBalance) = borrowBalanceStoredInternal(account);
211         if (mErr != MathError.NO_ERROR) {
212             return (uint(Error.MATH_ERROR), 0, 0, 0);
213         }
214
215         (mErr, exchangeRateMantissa) = exchangeRateStoredInternal();
216         if (mErr != MathError.NO_ERROR) {

```

```

217         return (uint(Error.MATH_ERROR), 0, 0, 0);
218     }
219
220     return (uint(Error.NO_ERROR), qiTokenBalance, borrowBalance,
exchangeRateMantissa);
221 }

```

```

277     /**
278     * @notice Return the borrow balance of account based on stored data
279     * @param account The address whose balance should be calculated
280     * @return (error code, the calculated balance or 0 if error code is non-zero)
281     */
282     function borrowBalanceStoredInternal(address account) internal view returns
(MathError, uint) {
283         /* Note: we do not assert that the market is up to date */
284         MathError mathErr;
285         uint principalTimesIndex;
286         uint result;
287
288         /* Get borrowBalance and borrowIndex */
289         BorrowSnapshot storage borrowSnapshot = accountBorrows[account];
290
291         /* If borrowBalance = 0 then borrowIndex is likely also 0.
292         * Rather than failing the calculation with a division by 0, we
immediately return 0 in this case.
293         */
294         if (borrowSnapshot.principal == 0) {
295             return (MathError.NO_ERROR, 0);
296         }
297
298         /* Calculate new borrow balance using the interest index:
299         * recentBorrowBalance = borrower.borrowBalance * market.borrowIndex /
borrower.borrowIndex
300         */
301         (mathErr, principalTimesIndex) = mulUInt(borrowSnapshot.principal,
borrowIndex);
302         if (mathErr != MathError.NO_ERROR) {
303             return (mathErr, 0);
304         }
305
306         (mathErr, result) = divUInt(principalTimesIndex,
borrowSnapshot.interestIndex);

```

```

307         if (mathErr != MathError.NO_ERROR) {
308             return (mathErr, 0);
309         }
310
311         return (MathError.NO_ERROR, result);
312     }

```

```

334     /**
335      * @notice Calculates the exchange rate from the underlying to the QiToken
336      * @dev This function does not accrue interest before calculating the exchange
337      * rate
338      * @return (error code, calculated exchange rate scaled by 1e18)
339      */
340     function exchangeRateStoredInternal() internal view returns (MathError, uint)
341     {
342         uint _totalSupply = totalSupply;
343         if (_totalSupply == 0) {
344             /*
345              * If there are no tokens minted:
346              * exchangeRate = initialExchangeRate
347              */
348             return (MathError.NO_ERROR, initialExchangeRateMantissa);
349         } else {
350             /*
351              * Otherwise:
352              * exchangeRate = (totalCash + totalBorrows - totalReserves) /
353              * totalSupply
354              */
355             uint totalCash = getCashPrior();
356             uint cashPlusBorrowsMinusReserves;
357             Exp memory exchangeRate;
358             MathError mathErr;
359
360             (mathErr, cashPlusBorrowsMinusReserves) = addThenSubUInt(totalCash,
361                 totalBorrows, totalReserves);
362             if (mathErr != MathError.NO_ERROR) {
363                 return (mathErr, 0);
364             }
365
366             (mathErr, exchangeRate) = getExp(cashPlusBorrowsMinusReserves,
367                 _totalSupply);

```



```
363         if (mathErr !== MathError.NO_ERROR) {  
364             return (mathErr, 0);  
365         }  
366  
367         return (MathError.NO_ERROR, exchangeRate.mantissa);  
368     }  
369 }
```

Status

✓ Fixed

[WP-L4] Replace imprecise `exchangeRateStored()` redemption calculation with direct `redeemUnderlying()` call

Low

Issue Description

Using `exchangeRateStored()` to compute cTokens to redeem for an intended underlying amount is imprecise and can revert if stale rate causes an over-estimate.

Under/over redemption, potential revert if computed cTokens exceed balance after accrual.

Consider adding `redeemUnderlying` method to IBENQI:

```
function redeemUnderlying(uint redeemAmount) external returns (uint);
```

Then replace conversion + redeem with `redeemUnderlying(_amount)` .

<https://github.com/treehouse-gaia/tAVAX-protocol/blob/181a25d9ad126b13eabb9957ba3e7f828b5a6e23/contracts/strategy/actions/benqi/BenqiWithdraw.sol#L51-L74>

```
51     function _withdraw(uint _amount, uint16 _poolId) internal returns (uint, bytes
    memory) {
52         address _lendingPool =
    IProtocolPoolController(PROTOCOL_CONTROLLER).getPoolAddress(PROTOCOL_ID, _poolId);
53         uint actualWithdrawn;
54
55         if (_lendingPool == qisAVAX) {
56             // For qiSAVAX withdrawal, redeem qiSAVAX tokens to get sAVAX
57             // Calculate the qiSAVAX tokens to redeem based on the exchange rate
58             // Note: This assumes the strategy has enough qiSAVAX balance
59             uint exchange_rate = IBENQI(_lendingPool).exchangeRateStored();
60             _amount = (_amount * 1e18) / exchange_rate;
61             uint tokenBefore = IERC20(sAVAX).balanceOf(address(this));
62             uint result = IBENQI(_lendingPool).redeem(_amount);
63             require(result == 0, 'Benqi withdraw failed');
64             uint tokenAfter = IERC20(sAVAX).balanceOf(address(this));
65             actualWithdrawn = tokenAfter - tokenBefore;
```

```
66     } else {  
67         // For other qiTokens, handle ERC20 withdrawal  
68         // Note: This would need to be implemented for specific underlying tokens  
69         revert('Non-sAVAX withdrawal not implemented');  
70     }  
71  
72     bytes memory logData = abi.encode(_lendingPool, actualWithdrawn);  
73     return (actualWithdrawn, logData);  
74 }
```

Status

✓ Fixed

[WP-L5] NavBenqi does not handle error codes returned by IBENQI.getAccountSnapshot() , potentially misusing invalid values

Low

Issue Description

Unlike common contracts that directly revert on errors, Compound v2 style contracts typically don't revert but return error codes for callers to handle.

Callers should check the error code and revert when it's not a success code.

<https://github.com/treehouse-gaia/tAVAX-protocol/blob/181a25d9ad126b13eabb9957ba3e7f828b5a6e23/contracts/modules/nav/NavBenqi.sol#L32-L96>

```

@@ 32,40 @@
41     function nav(
42         address _target,
43         address[] memory _assetTokens,
44         address[] memory _debtTokens
45     ) external view returns (uint _nav) {
46         _nav += _target.balance;
47
48         uint wip;
49
50         // Process asset tokens
51         for (uint i; i < _assetTokens.length; ++i) {
52             (, uint qiTokenBalance, , uint exchangeRate) =
53             IBENQI(_assetTokens[i]).getAccountSnapshot(_target);
54             wip = (qiTokenBalance * exchangeRate) / 1e18;
55
56             if (wip > 0) {
57                 uint rate = RATE_PROVIDER_REGISTRY.getRateInAvax(_assetTokens[i]);
58                 if (rate > 0) {
59                     _nav += Math.mulDiv(rate, wip, 1e18);
60                 }
61             }
62
63             // Process debt tokens - accumulate total debt first

```

```

64     uint totalDebt;
65     for (uint i; i < _debtTokens.length; ++i) {
66         (, , uint borrowBalance, ) =
IBENQI(_debtTokens[i]).getAccountSnapshot(_target);
67         wip = borrowBalance;
68
69         if (wip > 0) {
70             uint rate = RATE_PROVIDER_REGISTRY.getRateInAvax(_debtTokens[i]);
71             if (rate > 0) {
72                 totalDebt += Math.mulDiv(rate, wip, 1e18);
73             }
74         }
75     }
76
@@ 77,95 @@
96     }

```

```

197     /**
@@ 198,200 @@
201     * @return (possible error, token balance, borrow balance, exchange rate
mantissa)
202     */
203     function getAccountSnapshot(address account) external view returns (uint,
uint, uint, uint) {
204         uint qiTokenBalance = accountTokens[account];
205         uint borrowBalance;
206         uint exchangeRateMantissa;
207
208         MathError mErr;
209
210         (mErr, borrowBalance) = borrowBalanceStoredInternal(account);
211         if (mErr != MathError.NO_ERROR) {
212             return (uint(Error.MATH_ERROR), 0, 0, 0);
213         }
214
215         (mErr, exchangeRateMantissa) = exchangeRateStoredInternal();
216         if (mErr != MathError.NO_ERROR) {
217             return (uint(Error.MATH_ERROR), 0, 0, 0);
218         }
219

```

```
220         return (uint(Error.NO_ERROR), qiTokenBalance, borrowBalance,  
221                exchangeRateMantissa);  
    }
```

Status

✓ Fixed

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.



Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.